

INTERNAL RECRUITMENT MANAGEMENT SYSTEM

Developer Intern Technical Assignment

Next.js + Node.js + PostgreSQL | Internal Training / R&D Project

1. Assignment Purpose

Build an internal recruitment management system using fictional data and fictional business rules. This is a training/R&D assignment and must not use client data, client workflows, client branding, client documents, client APIs, credentials, or production infrastructure.

The assignment evaluates full-stack development, authentication, RBAC, database design, REST APIs, dashboards, file uploads, audit logging, testing, documentation and CI.

2. Project Objective

Create a recruitment platform where HR Managers create jobs, Recruiters manage candidates and applications, and Applicants browse jobs and apply. The system must use scalable permission-based authorization so new roles can be added later without rewriting security logic.

3. Roles

Role

Responsibilities

HR Manager

Create/manage jobs, view recruitment statistics, manage recruiters, view candidates and applications.

Recruiter

Manage assigned jobs, create candidate profiles, upload resumes, move candidates through the recruitment pipeline.

Applicant

Create profile, browse jobs, apply, and view own application status.

System Admin

Manage users, roles and audit logs for administration/testing.

4. Functional Modules

Module

Required Features

Priority

Authentication

Register, login, logout, JWT, password hashing.

P0

RBAC

Role + permission based access enforced by backend.

P0

Job Management

Create, edit, publish, close and list jobs.

P0

Candidate Management

Create/edit profile, search/filter candidates.

P0

Resume Upload

Secure resume upload and access.

P0

Applications

Apply, list applications, update status.

P0

Recruitment Pipeline

Applied → Screening → Interview → Selected/Rejected.

P0

Dashboard

Recruitment metrics and basic charts.

P1

Audit Logs

Track important actions/status changes.

P1

Notifications

Basic in-app application notifications.

P1

Admin

User/role management and audit viewer.

P1

5. Recruitment Workflow

Job: DRAFT → PUBLISHED → CLOSED

Application: APPLIED → SCREENING → INTERVIEW → SELECTED
↘ REJECTED

The backend must validate valid status transitions. Users must not be able to set arbitrary application statuses through an API request.

6. Example User Journey

HR Manager logs in.

HR Manager creates a fictional job such as Full Stack Developer.

HR Manager publishes the job.

Applicant registers and completes their profile.

Applicant searches for the job and applies.

Recruiter sees the application.

Recruiter moves it to Screening.

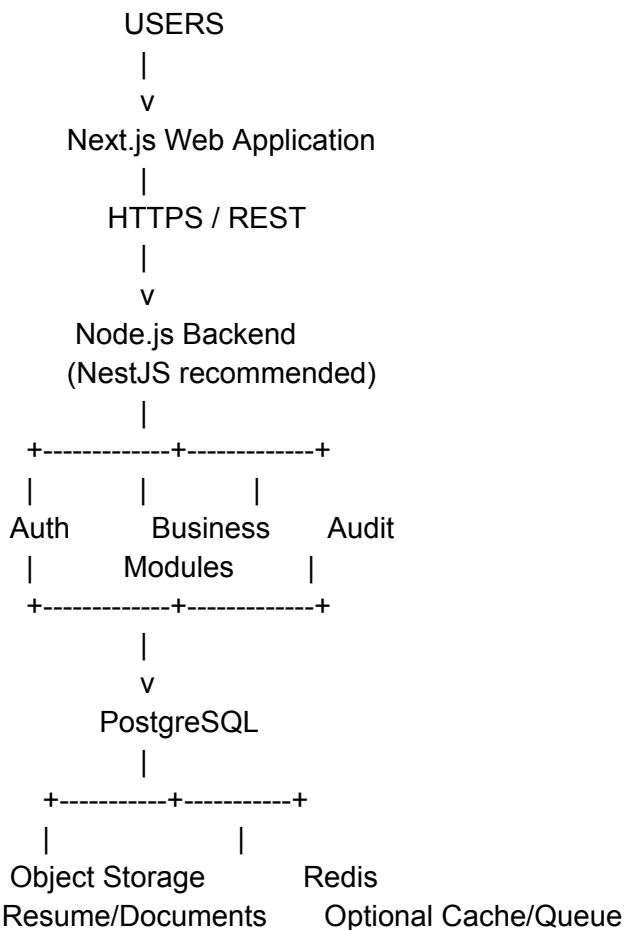
Recruiter records an interview.

Recruiter marks the candidate Selected or Rejected.

Applicant sees the latest status.

Audit log records important changes.

7. Technical Architecture



8. Technology Stack Layer

Technology
Requirement

Frontend
Next.js + TypeScript
Required

UI
Tailwind CSS
Required

Forms
React Hook Form + Zod
Recommended

Backend
Node.js + NestJS + TypeScript
Required

API
REST + JSON
Required

API Docs
Swagger/OpenAPI
Required

Database
PostgreSQL
Required

ORM
Prisma
Required

Authentication
JWT + refresh tokens
Required

Password
Argon2 or bcrypt
Required

Storage

Local development + object-storage-ready design
Required

Testing
Jest + Supertest + Playwright
Required

Version Control
Git + GitHub
Required

CI/CD
GitHub Actions
Required

Containers
Docker + Docker Compose
Recommended

10. Database Design

Table
Important Fields
Purpose

users
id, email, password_hash, role_id, status
User accounts

roles
id, name
System roles

permissions
id, key, description
Fine-grained permissions

role_permissions
role_id, permission_id
Role-to-permission mapping

jobs
id, title, description, location, type, status, created_by
Job openings

candidates

id, user_id, name, phone, skills, resume_key

Candidate profiles

applications

id, job_id, candidate_id, recruiter_id, status

Applications

application_events

id, application_id, from_status, to_status, changed_by

History

interviews

id, application_id, scheduled_at, interviewer_id, notes, status

Interviews

notifications

id, user_id, type, title, message, read_at

Notifications

audit_logs

id, actor_id, action, entity_type, entity_id, metadata, created_at

Audit trail

11. RBAC Requirement

Do not hard-code authorization everywhere as role checks. Build reusable permission-based authorization.

Example permissions: job.create, job.update, job.publish, candidate.view, candidate.update, application.view, application.status_update, audit.view.

Roles map to permissions.

Backend guards/middleware enforce permissions.

Frontend can hide unavailable actions for UX, but backend authorization is the security boundary.

The intern must demonstrate how a new role can be added by configuring permissions rather than rewriting modules.

12. Core REST APIs

Method

Endpoint

Purpose

POST

/api/auth/register

Register

POST

/api/auth/login

Login

POST

/api/auth/refresh

Refresh token

POST

/api/auth/logout

Logout

GET

/api/jobs

Search/list jobs

POST

/api/jobs

Create job

PATCH

/api/jobs/:id

Update job

POST

/api/jobs/:id/publish

Publish job

POST

/api/jobs/:id/close

Close job

GET

/api/candidates

Search/list candidates

POST

/api/candidates

Create candidate

PATCH

/api/candidates/:id

Update candidate

POST

/api/candidates/:id/resume

Upload resume

POST

/api/jobs/:id/apply

Apply

GET

/api/applications

List applications

PATCH

/api/applications/:id/status

Change application status

POST

/api/applications/:id/interviews

Create interview

GET

/api/notifications

Get notifications

PATCH

/api/notifications/:id/read

Mark notification read

GET

/api/audit-logs

View audit logs

13. Frontend Pages

Area

Pages

Public

/, /jobs, /jobs/[id], /login, /register

HR Manager

/hr/dashboard, /hr/jobs, /hr/jobs/new, /hr/candidates, /hr/applications, /hr/recruiters

Recruiter

/recruiter/dashboard, /recruiter/jobs, /recruiter/candidates, /recruiter/applications,
/recruiter/interviews

Applicant

/applicant/dashboard, /applicant/profile, /applicant/jobs, /applicant/applications

Admin

/admin/dashboard, /admin/users, /admin/roles, /admin/audit-logs

14. Dashboard Requirements

Total active jobs.

Total applications.

Applications by status.

Interviews scheduled.

Selected and rejected candidates.

Recent application activity.

At least one useful chart.

Dashboard API should return aggregated metrics rather than forcing the frontend to calculate everything.

15. Resume Upload

Allow PDF, JPG, JPEG and PNG for MVP.

Maximum file size should be configurable, e.g. 5–10 MB.

Validate MIME type and extension.

Generate a unique storage key.

Store metadata in the database.

Keep production files private.

Use signed/authorized URLs.

Never expose storage credentials to the frontend.

16. Security Requirements

Hash passwords using Argon2 or bcrypt.

Use JWT access tokens with refresh-token strategy.

Validate all write requests.

Use ORM/parameterized queries.

Rate-limit login and sensitive APIs.

Restrict CORS.

Never commit secrets or .env files.

Never log passwords, tokens or sensitive document contents.

Enforce object-level authorization so changing an ID in a URL cannot expose another user's data.

Create audit logs for important actions.

17. Testing Requirements

Successful registration/login.
Invalid login.
Protected API without token.
Protected API with insufficient permission.
HR Manager can create a job.
Applicant cannot create a job.
Applicant can apply to a published job.
Applicant cannot apply twice to the same job.
Invalid application status transition is rejected.
Resume upload accepts valid files and rejects invalid files.
User cannot access another candidate's private resume.
Audit log is created after an important status change.
Expired/invalid refresh token is rejected.

18. Development Phases

Phase

Task

Expected Output

1

Project setup

Next.js, NestJS, PostgreSQL, Prisma, Docker, README

2

Authentication

Register/login/logout, JWT, hashing

3

RBAC

Roles, permissions, guards

4

Job module

CRUD, publish/close, public listing

5

Candidate module

Profile, search/filter, resume

6

Application module

Apply, list, status workflow, history

7

Interview module

Schedule/update interviews

8

Dashboards

HR, Recruiter, Applicant dashboards

9

Notifications + Audit

Notifications and audit trail

10

Testing + CI

Unit/API/E2E tests and GitHub Actions

11

Final review

Documentation, demo, cleanup

19. Git & Code Quality

Use feature branches.

Use meaningful commit messages.

Use pull requests.

Run lint, tests and build before PR.

Keep business logic out of React components.

Keep controllers thin; put business logic in services/use cases.

Use DTOs and validation.

Avoid duplicated code.

Use meaningful names.

Document important technical decisions.

20. Required Documentation

README with setup instructions.

Environment variable documentation.

Architecture diagram.

ER/database diagram.

Swagger API documentation.

RBAC permission matrix.

Application status transition diagram.

Testing instructions.

Deployment notes.

Known limitations and future improvements.

21. Expected Deliverables

Deliverable

Expected Result

Source Code

Complete Next.js frontend + Node.js backend

Database

PostgreSQL schema + Prisma migrations + seed data

Authentication

Secure login/register/logout + JWT

RBAC

Role + permission authorization

Job Management

Working job creation/publishing

Candidate Management

Profile + search/filter + resume upload

Applications

Recruitment pipeline

Dashboard

HR/Recruiter/Applicant views

Audit

Status/action history

Testing

Automated tests for core workflows

CI

GitHub Actions lint/test/build

Documentation

README + architecture + API + database docs

Demo

5–10 minute walkthrough

22. Definition of Done

All P0 modules work end-to-end.

Backend authorization is enforced.

Application status transitions are validated.
Private candidate data cannot be accessed by unauthorized users.
Core automated tests pass.
Application can be started locally using documented commands.
Swagger is available.
No secrets are committed.
README is complete.
GitHub Actions passes lint/test/build.
Intern can explain architecture and technical decisions during review.

23. Evaluation Criteria

Area

Weight

Functionality & requirements
25%

Code quality & architecture
20%

Backend/API design
15%

Database design
10%

Security & RBAC
10%

Frontend/UI/UX
10%

Testing
5%

Documentation & presentation
5%

24. Confidentiality Rule

This is an internal training/R&D project. The intern must use fictional company names, fictional candidates and synthetic data. Do not copy client requirements, workflows, screenshots, designs, documents, credentials, APIs, source code, database schemas or business rules into this assignment.

25. Final Technical Challenge

Add a new role called Interviewer without rewriting the existing authorization framework. The intern should create the role, assign appropriate permissions, protect the interview APIs, and demonstrate the new role in the final presentation.

END OF INTERNAL DEVELOPER INTERN ASSIGNMENT